

Scaling Virtual Worlds: A Distributed Server Architecture for Real-Time Multiplayer Games

June Yoo
june.yoo@yale.edu

Advisor: Michael Shah
michael.shah@yale.edu

*A Senior Thesis as a partial fulfillment of requirements
for the Bachelor of Science in Computer Science*

Department of Computer Science
Yale University
May 1, 2025

Acknowledgements

I want to express my gratitude to Dr. Mike Shah, whose continued support and guidance made this project possible. I would also like to thank Professor Y. Richard Yang, Xiao Shi, and Scott Pruett for inspiring my curiosity for this project as part of their course on Distributed Systems. Finally, I want to thank the Yale Department of Computer Science for for the opportunities and resources that have enriched my time at Yale.

Contents

1	Introduction	5
2	Background	6
2.1	Contemporary Game Servers	6
2.1.1	Approaches to Scaling	6
2.2	Motivation for Our Approach	7
2.2.1	Consistency vs. Availability	8
2.2.2	Fault/Partition Tolerance	8
2.2.3	In-Game Spatial Locality	9
3	Methodology	10
3.1	Architecture	10
3.1.1	Object Store and Replica Manager	11
3.1.2	Object Locator	11
3.2	Optimizations	12
3.2.1	Connection Management	12
3.2.2	Match Prioritization	13
3.2.3	Event-Driven Messaging	13
3.3	Implementation	14
3.4	Experimental Design	14
4	Results	15
5	Related Work	17
6	Conclusions	19
7	Future Work	20
7.1	Missing Features	20
7.2	Further Testing	20
7.3	Assumptions and Limitations	20
A	Experimental Parameters	24

Scaling Virtual Worlds: A Distributed Server Architecture for Real-Time Multiplayer Games

June Yoo

Abstract

Modern real-time games rely on heavily centralized server architectures, avoiding the tiered distributed systems common in other online software to minimize end-user latency. Instead, clients typically communicate directly with the servers that manage the game state, thereby approximating real-time interaction. However, it is well-known that such single-server designs inevitably incur the cost of scalability, making them fundamentally incompatible with what we see as a growing demand for large virtual worlds in spaces like virtual reality. In this project, we explore whether it is possible to create a distributed architecture that can scale virtual worlds while maintaining the illusion of a single instance to the player. To this end, we review several existing scaling techniques, finding that they each leverage several unique properties of games. In particular, we note that games, in contrast with other distributed applications, generally display strong spatial locality and tolerance to transient inconsistency. These patterns inform the design of our architecture, which—building upon the most current approach in the literature—partitions ownership of the game state between the participating nodes by game object, then allows them to selectively retain relevant information through a proximity-based discovery mechanism. We also introduce several novel optimizations, one of which allows us to confirm the correctness of our protocol. We encapsulate this architecture into the first reusable software library built for distributed networking in games. Finally, we implement and test various distributed deployments of a game using our new library, finding that it successfully scales the game well beyond the single-server baseline without sacrificing other areas of performance.

1 Introduction

When a player launches an online multiplayer game, they typically control an *avatar* within a shared *game world*. This world may contain some modifiable objects as well as avatars controlled by other players, and—in the case of real-time games like shooters or virtual reality simulations—each of these players must be able to responsively read and write to a consistent game state as if they were interacting with each other concurrently. As one might quickly deduce, such games impose stringent performance requirements due to their tight deadlines and write-heavy workloads. Hence, in almost all multiplayer games today, all clients communicate with a single authoritative server, minimizing the need to pass data back and forth across networks. Accordingly, the most popular real-time games (*Fortnite*, *PUBG*, *CoD*) remain severely limited in capacity to the current day, supporting no more than on the order of 100 players per instance.

However, as of late, games are evolving from small intimate experiences to large communal ones, with the biggest titles managing millions of concurrent players. Moreover, there is growing demand for scalable virtual worlds that facilitate real-time interaction as major companies pursue mainstream adoption of virtual reality and the “metaverse.” These developments suggest the possibility of moving beyond single-server architectures and scaling games using distributed architectures. In this project, inspired by the zeitgeist surrounding decentralized technology, Bluesky/Mastodon, and the AT Protocol for distributed social networks, we explore a similar type of architecture: a fully distributed game instance which has no dependency on any centralized authority.

Previous research in this field has been limited. It is possible that the tight development cycles in the gaming industry have not yielded the engineering capacity for the complex and idiosyncratic solutions that would be necessary. However, some theoretical solutions have been proposed, the most well-known of which is *Colyseus*. [1] This design splits ownership of the game state between participating nodes, and each node automatically subscribes to relevant game state through a decentralized discovery mechanism.

With this project, I explore modern game servers, identifying limitations and common patterns. Informed by this review, I make several improvements to the original *Colyseus* protocol, then use it to implement a networking library for real-time games. This library enables several new server deployment patterns currently uncommon in games: for instance, a distributed network of edge servers could support a single, larger game instance which enables all players of the game to interact with each other in real time. Furthermore, players would also be able to host their own servers for large-scale games in a federated fashion without straining or depending on a centralized maintainer, allowing games to outlive their publishers. I test the feasibility of such scenarios through local simulations, finding that the architecture is indeed suitable for increasing total bandwidth by orders of magnitude.

2 Background

In this section, we explore the existing approaches to game server implementation and scaling, then extend upon those ideas to motivate a specific distributed architecture for scaling virtual worlds.

2.1 Contemporary Game Servers

In software, games are typically represented as a collection of *game objects*, which change at a consistent frequency (or *tickrate*) according to an *update function* acting on their own state and the states of other objects in their vicinity. Moreover, we already introduced the concept of a game *world* in the introduction, in which all of these objects coexist and interact with each other.

Considering the constant flow of data necessary between objects, perfect synchronous play over a network is inherently impossible due to the speed-of-light limit, but is emulated by minimizing network overhead. Thus, in order to facilitate direct client-server communication, most games dedicate one machine per match.

Moreover, games make many complex optimizations in order to further reduce the cost incurred by network latency. A common approach, for instance, is to use delta-encoded updates by game object over UDP, and sending occasional full snapshots of the game state to ensure that clients stay up to date. To account for the transient inconsistencies that this approach may introduce, game clients use techniques such as interpolation and extrapolation to ensure a consistent and smooth gameplay experience for the player while keeping the information loosely aligned with what is present on the server.

2.1.1 Approaches to Scaling

Traditionally, game worlds have been partitioned into discrete sections called *maps* or *zones* for reasons of local performance, such as resource loading.¹ This is particularly common in MMORPGs (Massive Multiplayer Online Role-Playing Games), in which the core gameplay mechanic involves players exploring large, expansive worlds. [2] This partition thus naturally lends itself to some of the simplest approaches to distributed deployment: individual machines host certain zones and are limited in player capacity, refusing any connections beyond the fixed limit. As players move between zones, their connections are handed off to a different machine. A top-level orchestration system often

¹Games that go against this paradigm have long existed but are becoming increasingly popular, providing us with yet another motivation for alternatives; for instance, *The Legend of Zelda: Breath of the Wild*, released in 2017, is frequently cited as a revolutionary development.



Figure 2.1: **Left:** Dozens of players gather in one in-game zone in *Club Penguin* (2005). To either side are paths which transport players to other zones. **Right:** All the zones visible on a map as part of a single, apparently continuous world.

performs dynamic load balancing, scaling the number of servers assigned to each map as required by the concurrent player count and density.

In the current day, this approach is an ubiquitous game development idiom in itself. In addition, modern games often partition by geographic (real-world) region, similarly to other distributed systems, as real-time interaction across large physical distances will necessarily feel unresponsive due to the speed-of-light limit. [3]

Other games scale by duplicating the game world as needed. *Fortnite* is a game in the *Battle Royale* genre which supports up to 100 players in any given instance of the game, each deployed on a single 4.8xlarge AWS instance [4]. The number of instances is then scaled up and down with the number of games in progress, with any persistent data (such as player rank) stored separately in a traditional distributed database. Using this method, *Fortnite* reached up to 3.4 million concurrent users in 2018, before failing due to bottlenecks in auxiliary systems unrelated to gameplay, showing that it is possible to achieve nearly unlimited horizontal scaling when individual game instances require little to no communication between each other.

2.2 Motivation for Our Approach

Despite the success of such approaches, no commercial attempt has been successful in distributing a single world across multiple machines while maintaining the illusion of a single application. Noticing that none of these techniques are specific to single-server architectures, we aim to create a distributed version of this approach to allow for pseudo-“vertical” scaling. This is in contrast to strict vertical scaling—which relies on improving individual machines—and horizontal scaling, as described in 2.1.

Any game server must be designed around several challenges which for implementing fast multiplayer games. First, real-time gameplay must feel fast and responsive, which requires minimizing staleness and serializing updates as fast as possible. Secondly, games are computationally expensive, requiring us to run physics, collision, and other complex simulations on many objects at the game’s internal framerate.

Distributed systems remove traditional limits of scale imposed by single-server archi-

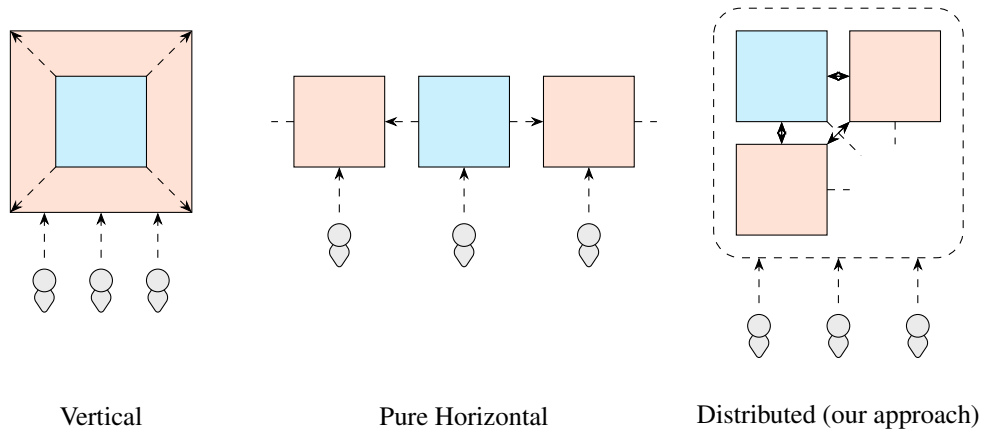


Figure 2.2: Three approaches to scaling a system, with original machines in blue and new in red. The first two are commonly employed in modern games.

tures but come with their own inherent drawbacks. In particular, the CAP theorem in distributed systems shows that it is impossible for any distributed system to achieve consistency, availability and partition safety guarantees simultaneously. Thus, the specific design of our distributed system requires us to examine some properties of real-time games and align the engineering tradeoffs accordingly.

2.2.1 Consistency vs. Availability

For instance, one property of games is that a limited amount of transient inconsistency is acceptable, as unavoidable delays due to latency and the speed of human perception are on a similar order of magnitude. Furthermore, games can use interpolation or extrapolation at the application level to mask inconsistency in sane ways because game data typically exhibits spatial locality (described in 2.2.3). For instance, the client can usually expect an object moving in one direction to continue moving in that direction. When the client receives newer data from the server, it can interpolate the object’s position towards the correct location without sacrificing visual continuity.

On the other hand, availability is of critical importance to game servers. Each read and write event between client and server is expected to happen near-instantaneously and continuously; any delays or inconsistency in this process are likely to be noticed by the player and ruin an otherwise seamless gaming experience.

Hence, in designing and implementing our architecture, our primary motivation is that by distributing the computationally heavy task of updating game state between servers, we can maintain reasonable availability at high workloads while sacrificing a minimal level of consistency.

2.2.2 Fault/Partition Tolerance

Distributed systems should be tolerant to failures so that errors on one node do not result in catastrophic outcomes for the system as a whole. This is especially important in games,



Figure 2.3: Illustration of spatial locality in *Minecraft* (2011): the game omits rendering distant regions unlikely to be interacted with by the player.

where a game-wide failure may have pronounced impacts on related systems including rewards or matchmaking, ultimately lowering player retention.

Many distributed systems also explicitly implement a failure recovery mechanism, allowing the failed node to rejoin the system. In this case, however, we posit that reliable mid-game failure recovery is at most application-dependent in terms of its usefulness. For instance, if a node fails while creating a bullet in mid-flight, it is unlikely that we would want the bullet to spontaneously reappear when the node rejoins the network. In other cases, the game state may have simply progressed too far without the absentee node, and the remaining nodes may not care about the rejoining node's stale data. It is possible to mitigate such cases using some common strategies in distributed systems; for instance, the remaining nodes may choose to split among themselves the responsibilities of a node when they detect that it has failed. However, in the general sense, durability in games is handled optimistically, with the assumption being that any gameplay that happens prior to a failure is far too stale by the point of recovery and can be lost.

2.2.3 In-Game Spatial Locality

One assumption leveraged by both the servers described in 2.1 and the approaches described in 2.1.1 is that games typically exhibit strong spatial locality; that is, each game object is typically only concerned with objects in its immediate vicinity (called its *area of interest*) in order to update itself. Moreover, game objects tend to stay in similar regions of the world for long periods of time and have limited mobility, whereas in large social networks, any two resources may be connected by at most a few hops. We make this a central part of the design of our server architecture: that is, a node should only need to communicate with other nodes in the system that contain information which is likely to be relevant to its own.

3 Methodology

In the following section, we describe the overall architecture of our proposed distributed game server, as well as discussing several optimizations and guarantees. The design is heavily based on the original Colyseus paper, with several modifications for optimization or practicality. For the purposes of verifying and experimenting with this architecture, we have implemented the underlying protocol as a software library, abstracting away the logic needed for distributed networking when building games. This library implements a distributed "middle-layer" while exposing normal client-server communication APIs such that servers and clients can still pretend to talk to each other as normal.

In our design, we assume that the set of objects any given object needs to update itself lives entirely within its area of interest, as per 2.2.3: for instance, a projectile must be within some bounding box of a player for collisions to occur between the two. For globally pertinent objects, the area of interest can be defined to span the whole playable area on the application side.

3.1 Architecture

Our distributed server consists of some configuration of one or more nodes. The nodes are flatly structured, such that every node can communicate freely with all other nodes in the network, although they will not necessarily have to do so.

Each node has a local *object store* consisting of *primary objects* and *replicas*. Each primary object in the game world is owned by a unique node in the system, which manages updates to that object, ensuring quorumless writes to that object. Other nodes can own a replica of the primary which is weakly consistent with the primary, but cannot directly commit updates to it; all updates are forwarded to the primary copy, which serializes the final update order and redistributes it to its replicas.

Each node consists of two components: a *replica manager*, which manages loosely synchronized replicas of game objects on each node, and an *object locator*, which enables nodes to subscribe to certain regions of the game world through range queries and be notified when other nodes publish game objects there. The original Colyseus architecture also proposes an *object placer*, which reassigns primary objects to nodes using a load-balancing algorithm. In both the paper and this project, the object placer is not implemented, leaving all primary objects attached to their node of origin. We posit that it is likely more optimal for placement to be handled at the application level, as it is difficult to formulate a general heuristic that will handle all use cases effectively.

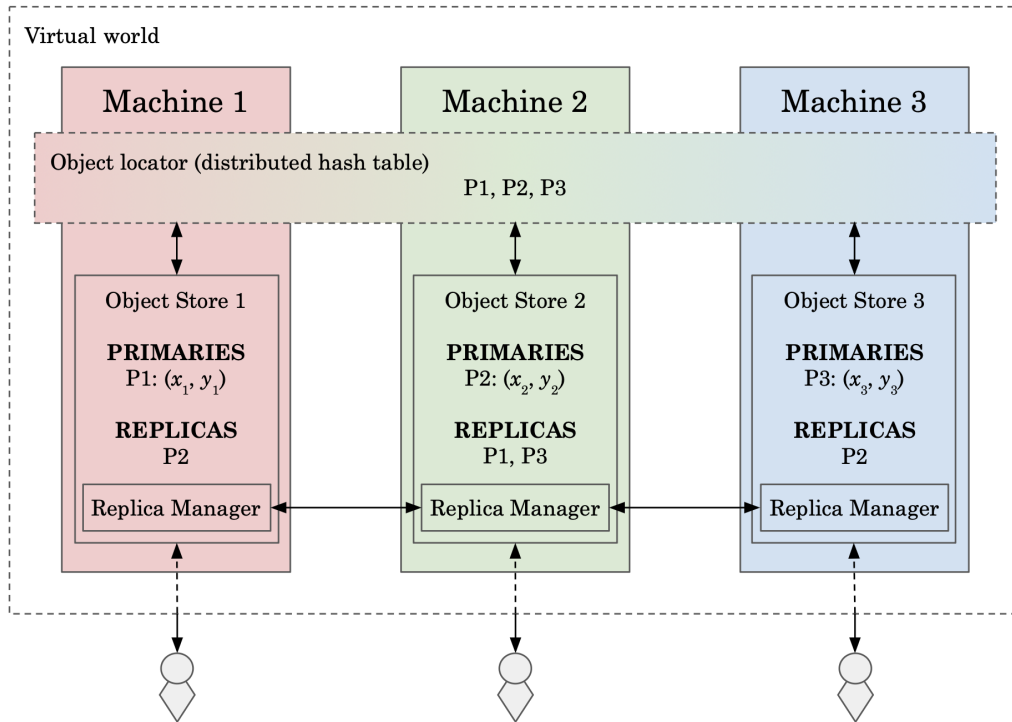


Figure 3.1: Simple diagram of a deployment with three nodes, each owning one primary object. P1 and P3 are both within range of P2, but not of each other. The users see the illusion of one virtual world.

3.1.1 Object Store and Replica Manager

Managing the primary objects of a node largely follows the traditional game loop pattern exactly. The replica manager of a node maintains copies of objects that the primary objects are likely to need to update themselves in the near future, which is determined by a range query (such as a bounding box).

Each primary object publishes a range query to the object locator described in the next section, which forwards any matches to the relevant replica manager if found. From that point on, the replica manager directly corresponds with the owner of the primary copy to receive updates to its replicas. At set intervals, the replica manager checks each of its replicas to verify that they are still within its primaries' areas of interest, and discards replicas for which the condition is not met.

Each object can receive updates and other network calls from neighboring nodes asynchronously. The objects then synchronize the incoming calls through concurrency controls and, in the case of the primary object, serialize them into a final ordering, remaining the single source of truth for the object.

3.1.2 Object Locator

The object locator acts as a distributed discovery service for nodes to find other object which their primary objects are interested in. Essentially, it isolates In the original pa-

per, Colyseus tests both a traditional distributed hash table and a range-query distributed hash table as an object locator, finding that the range-query hash table is more effective. In particular, the locator is implemented using Mercury [5], a sophisticated distributed hash table that supports multi-attribute range-based queries and performs its own load balancing. For the scope of this project and the experiments, we choose to simplify the object locator as a monolithic range-based lookup service that all servers use as a single source of truth. Our solution is nontrivial, requiring the service to maintain a quadtree data structure against concurrent publications and subscriptions, but reintroduces a single bottleneck, preventing the resulting system from being fully decentralized. Nonetheless, it can be considered an abstraction for a decentralized alternative, as the replica manager interfaces with the locator in a way that is agnostic to its underlying implementation.

To prevent frequent publications and subscriptions, each entry sent to the locator has a TTL which is an adjustable parameter of the system, allowing for optimistic matches to occur even when they are not exact; the node which receives the match can then decide whether it is truly interested in the object. Furthermore, both publications and subscriptions are based on the *predicted area-of-interest* of the object, which uses its bounding box in combination with its current velocity to declare an expanded region of the map it is likely to be interested in during the next second, eagerly fetching objects in that area.

3.2 Optimizations

We add several optimizations on top of the original Colyseus protocol.

3.2.1 Connection Management

Recall that all nodes in our network must be able to communicate with all other nodes. To facilitate this process, every game object holds the address of the node that owns its primary copy, so that any node can directly contact the owner for updates when it encounters the object. This also means that nodes no longer need to know about the shape of the full configuration or neighbors' addresses, allowing the deployment to scale up and down in a flexible manner.¹

Each replica manager has a list of its “active” connections and a map from its replicas/primaries to their corresponding nodes. Batching network calls by node can significantly reduce the number of overall requests, especially for well-optimized deployments in which nodes roughly correspond to in-game geographic regions. We implement batching at the internal tickrate when primaries send updates to their replicas, but do not do so for other requests, noting that this may cause the overall execution to be delayed by several frames in bad scenarios.

¹However, the application would need to implement object placement to retain the lost data or make use of the added throughput.

3.2.2 Match Prioritization

In our implementation, we prioritize delivering information to a subscriber node as soon as we know that a match has occurred in the locator, doing so through two different mechanisms.

When the locator first detects a match between a publication and subscription, the subscriber node must still contact the publisher node in order to initialize its replica. To bypass this additional network call, we provide the locator with full information about all published objects, and the locator forwards new subscribers with the latest copy if a match is found. Although this is space-redundant, this approach remains efficient as the locator only holds static copies of the information.

Furthermore, the game may frequently create new objects during its execution; for instance, a new player may join the game world, or a player may fire a projectile. In the latter scenario, we would presumably want nodes subscribed to the player to know about the created projectile immediately, rather than having to wait for the parent node to publish to the locator and trigger a match. As such, we allow the application to tie newly created objects to some parent object. When the object is created, all nodes subscribed to the parent object will immediately and directly receive a replica of the newly created object, eliminating the object locator step.

3.2.3 Event-Driven Messaging

Diverging from the original design, we transition the core game logic to an event-driven model, which allows us to derive strong guarantees about consistency and fault tolerance.

We assume that our game is a deterministic state machine $(S, \mathcal{E}, s_0, f)$ such that each pairing of a state s in S (starting with the initial state, s_0) and event in $\mathcal{E}$ gives a new state $s' \in S$ under the game's update function $f : S \times \mathcal{E} \rightarrow S$. In our model, game state is partitioned by object, which is managed by the node that owns the primary copy of that object. Furthermore, we enforce that events in E must also act only on a single object; if some action impacts multiple objects in the game, we force each object to send an event to the next object that would be impacted in a chain-like fashion. Hence, as long as all clients and servers have identical implementations of f , the server provides s_0 of all its primaries upon connection, and the server correctly validates, serializes, and broadcasts events in E thereafter, all instances of each object will eventually be consistent, and therefore so will all instances of the game. This also eliminates the need to transmit entire states $s \in S \setminus \{s_0\}$ to clients, minimizing required bandwidth.²

We also notice that this model also naturally results in graceful degradation upon node failures, as described in 2.2.2, since each object ultimately depends only on itself for its state. That is, upon the failure of a server node, its associated primaries would simply stop publishing to the object locator and responding to updates, effectively allowing the rest of the game to continue without the objects in the failed node.

²This again becomes necessary when transmitting updates over UDP, although there are other methods of introducing safety (such as message ordering or built-in reliability). In this case, we may choose to add safety by sending occasional snapshots of the game state to clients or validating client state using checksums in the case of pathological inconsistencies.

3.3 Implementation

For this project, the above architecture is implemented as a library in Go, supporting a two-dimensional game world by default. Applications using the library must losslessly encode and decode all events in the game. Finally, they must communicate these events as well as the two-dimensional position and velocity of any relevant game objects to the library using the provided hooks. The library may periodically send requests to the application to apply new events to existing objects or “create” new objects (when discovered from another node), which the application must handle. We use gRPC and UDP to handle network calls between nodes.

For the scope of this project, we make several simplifications to the full protocol. Firstly, game objects hold server IDs rather than server addresses. This requires that deployments are fixed in size, as each node must hold a map of server IDs to addresses. However, as noted in 3.2.3, nodes can functionally leave and rejoin gracefully. Secondly, the object locator is left as an replaceable abstraction, as described in 3.1.2.

3.4 Experimental Design

To measure the effectiveness of our library, we build a small toy game and test it with various deployment configurations. In this game, each client controls a single avatar which can move continuously across the game world. These avatars are able to shoot projectiles that destroy the avatars of other users upon collision.

In our local deployments, each node hosts several clients regularly sending random inputs, and there is an artificial latency between the clients to the nodes as well as between participating nodes in order to simulate network conditions. Finally, each node also has an artificially induced maximum throughput. Precise parameters and details can be found in Appendix A.

We measure performance in terms of round-trip latency, keeping track of average and 99th percentile latency over a large number of requests. In several cases, the latency grows unboundedly due to *congestion collapse*, which we note in the results.

We compare several deployments with the single-server scenario. To ensure parity in all other aspects, we simply implement the server using our library, but as a single-node deployment. Since the object locator never returns matches during the program’s lifetime, the server simply serves the role of serializing updates as intended.

4 Results

We test a single-server deployment against deployments of our architecture using 2, 4, and 8 nodes. We then simulate increasing player populations of 4, 16, 24, and 32 players on each configuration, obtaining latency data for at least 100000 network calls from clients to servers.

The results, visualized in 4.1 and 4.2, broadly align with our expectations, showing a positive correlation between overall player capacity and the number of nodes. Specifically, the data seems to indicate that all configurations perform relatively similarly in sparsely populated scenarios but have different points of failure. Congestion collapse occurs after the 16-player case for the single-server configuration, but distributing the server to two nodes increases this to at least 24, and distributing to four increases it to at least 32. Hence, our architecture seems to successfully accomplish the goal of scaling a single virtual world well beyond the limitations of a single machine.

Importantly, there is also little to no variability in a single configuration's performance before its eventual collapse. For instance, the eight-node configuration experiences virtually no slowdown as its player population increases from 4 to 32. Thus, we so far have no evidence that our system introduces an unforeseen bottleneck, indicating that it may be possible to continue scaling overall capacity by orders of magnitude by adding more nodes to the system.

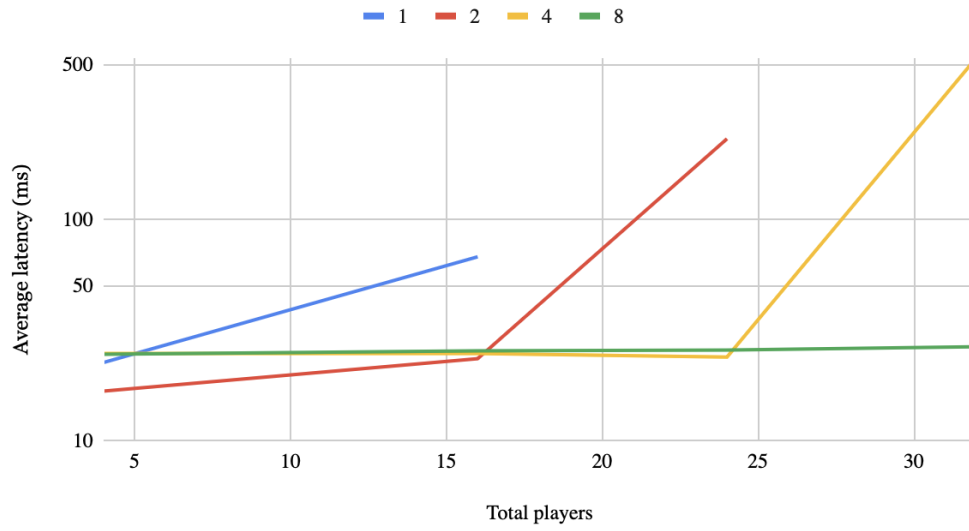


Figure 4.1: Average latency in ms (log scale) by player count for a client-server network call for configurations with 1, 2, 4, and 8 nodes ($n \geq 100000$ for each data point). Terminating lines indicate that the configuration collapsed under congestion past the data point, and latency did not stabilize.

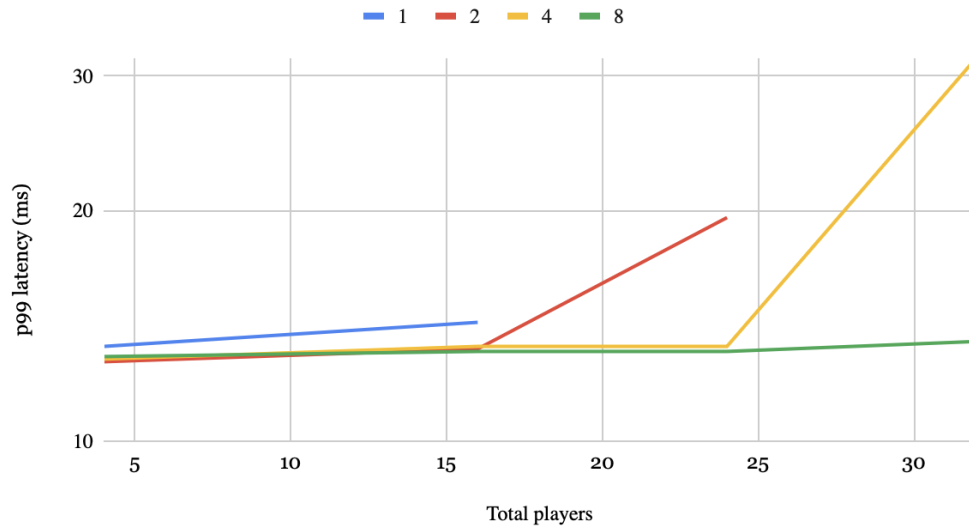


Figure 4.2: 99th percentile latency in ms (log scale) by player count ($n \geq 100000$).

5 Related Work

As previously mentioned, there has been limited research in the realm of distributing individual game instances as we have done above. In 2003, Butterfly.net [6] deployed the first “grid”-based approach, in which multiple servers were linked to work together as a virtual supercomputer. However, this naive approach was limited by bandwidth and the continued need for a central authority in the system, which would ultimately become a bottleneck.

Distributed systems such as *MASSIVE* [7] and *DIVE* [8] have demonstrated that it is possible to scale virtual reality to a large number of concurrent participants using IP multicast. However, these “virtual realities” consist of a combination of audio, video, and text streams, for which distributed system designs have largely diverged from those used for modern virtual reality and gaming. Today, virtual reality typically adopts the object-based organization of games.

More recent research that leverages the same assumptions about data locality described in 2.2.3. For instance, SimMud [9] is one of the first such architectures to be completely decentralized, not needing a single authority to dictate the state of the game. In this model, the game world is partitioned into a large number of small geographical regions known as *cells*, each of which is then responsible for its own game state. This allows for a much larger number of concurrent accesses, with experimental results showing up to 4000 players in a single game. However, this approach requires connection handoff as objects move across regions, making it unsuitable for fast-paced gameplay. Furthermore, SimMud is highly sensitive to population density, meaning that geographical regions in the game world that are activity hotspots ultimately become bottlenecks for the overall system.

In practice, this is similar in concept to the zone-based approach described in 2.1.1, except that the latter tightly couples in-game geography with network topology. This approach has been used extensively in combination with other scaling methods, particularly in MMORPGs. For instance, while *Ultima Online* (1997) first distributed a seamless open-world RPG into multiple parallel instances, calling each node a *shard*, games like *MapleStory* (2003) further fragmented each instance at least two more times (see 5.1). This sort of multi-layered fragmentation is common in distributed deployments of large games, and it is possible that one could incorporate the approach proposed in this project as another layer in such a multi-faceted architecture.

Nonetheless, for the purpose of distributing what are currently considered atomic game instances, Colyseus remains the most up-to-date and well-known architecture.

Independently of the work on distributed systems for games, there has been research on the correctness of distributed systems over unreliable networks. This is of particular importance since, while most distributed systems are built on reliable protocols such as



Figure 5.1: Upon logging into *MapleStory* (2003), players choose a “server” (right column). Each server is split into multiple “channels” (middle), which are all separate instances of the game that rely on the server’s database for persistent, non-real-time data such as character information. Finally, each channel is split by in-game zone.

TCP, protocols like UDP are foundational to low-latency gaming. Several approaches to ensuring correctness in the latter paradigm have been proposed, as in Muller *et al.*, [10] which relies on techniques borrowed from other distributed systems such as lag and time-warp. As mentioned in 3.2.3, our event-driven implementation is prone to failure without the reliability assumption, meaning that such research is relevant to prove that the guarantees hold in either case.

6 Conclusions

In this project, we explored the limitations of server architectures currently used in gaming and investigated whether it is possible to scale virtual worlds using a distributed design. We implemented one such potential architecture, improving on a previous design known as Colyseus, and converted it to a software library usable in the development of various games. Moreover, we implemented a game on top of this library and tested its performance using various distributed deployment configurations, finding that—without any noticeable side effects—we were able to meaningfully increase total system throughput and player capacity by adding more nodes.

This is a field for which little research currently exists, but these experiments signal that similar approaches may have the potential to scale virtual worlds to previously unprecedented sizes. Furthermore, once made production-ready (discussed in 7), this library would be the first of its kind, enabling game developers to distribute their servers without altering the underlying networking logic. Beyond the specific results of this project, we see growing demand for scalable virtual worlds, specifically in spaces such as gaming and virtual reality, and are hopeful that a more complete solution will arise in the near future.

7 Future Work

7.1 Missing Features

There is significant work to be done to make this library ready for production games; some of them have already been mentioned at various points throughout 3.

Otherwise, core features of networking libraries include reliability, security, and efficiency. In particular, this project does not attempt to encode messages into a maximally efficient binary format nor to verify incoming information at either end, which are both critical features in game networking.

Most game developers will also wish to have the option to switch between reliable and unreliable UDP as well as in-order and unordered messages for granular control over speed of communication. In our implementation, reliable communication is preferred in essentially all cases due to the adoption of an event-driven model. Nevertheless, the library should still implement some level of control over message ordering so that events can be delivered in strictly ordered sequences if necessary.

Providing observability tools will also be necessary so that developers can visualize the added value of distributing their game servers.

7.2 Further Testing

In this project, all testing was performed on a single machine, with artificial latency and bandwidth controls simulating a larger distributed system. These controls were adjusted conservatively to avoid the object locator or system CPU becoming the primary bottleneck. In further tests, we would like to deploy one node per machine in the intended configuration, eliminating the need for artificial controls.

One untested metric that could potentially impact gameplay is transient inconsistency, as discussed in 7.3. A possible future experiment could involve measuring the time delta between two objects entering each others' areas of interests from a local perspective and the objects being replicated on the corresponding servers.

7.3 Assumptions and Limitations

The above work makes several assumptions about the type of game being implemented using this library. Recalling 2.2.3, this project assumes that objects (and by extension, players) only need information about objects in their relative vicinity. However, certain

genres such as RTS (real-time strategy) often require players to quickly and spontaneously read and react to data from all parts of the game world.

Moreover, assuming that dynamic load balancing and object placement are implemented by the application, the above architecture is prone to extreme fragmentation as player density grows in in-game hotspots. For example, if thousands of players were gathered around an objective in a central location, one node would likely be insufficient to hold all of the relevant information, whether as primaries or replicas. One possible workaround would be to implement some sort of priority metric such that “important” objects like bosses or objectives can be discovered from far away, while nearby players are ignored when node resources are under strain.

Nodes in this network cannot guard against pathological or Byzantine failures. Therefore, this design is not appropriate for a federated deployment distributed between multiple independent parties, as this is likely to lead to cheating; instead, all nodes should be owned by the same central authority.¹ However, it could also be possible for very small groups to use this protocol to cooperatively host large games on local networks, relying on external factors such as social accountability to prevent cheating.

Finally, the guarantees about correctness in 3.2.3 only apply insofar as there being strong consistency between the eventual game states of all nodes. However, this does not mean that this resultant game state will be identical to the state had the game run on a single machine, or to the one intended by the developer. For instance, if the object locator fails to deliver information about a projectile in time to a node containing an object in its trajectory, the collision will simply never happen. Therefore, the game developer must be cautious about such scenarios so that no critical game events can be missed entirely when using this protocol.

¹Federated social networks, such as those running on the AT Protocol, suffer this same irresolvable issue—users are able to, for instance, arbitrarily change the timestamps on their posts. However, this is a larger practical problem in gaming, where users have stronger incentives to cheat.

Bibliography

- [1] Ashwin Bharambe, Jeffrey Pang, and Srinivasan Seshan. “Colyseus: a distributed architecture for online multiplayer games”. In: *Proceedings of the 3rd Conference on Networked Systems Design & Implementation - Volume 3. NSDI’06*. San Jose, CA: USENIX Association, 2006, p. 12.
- [2] Kuan-Ta Chen et al. “Game traffic analysis: an MMORPG perspective”. In: *Proceedings of the International Workshop on Network and Operating Systems Support for Digital Audio and Video. NOSSDAV ’05*. Stevenson, Washington, USA: Association for Computing Machinery, 2005, pp. 19–24. ISBN: 158113987X. DOI: 10.1145/1065983.1065988. URL: <https://doi.org/10.1145/1065983.1065988>.
- [3] Ibrahim Cevizci, Melike Erol, and Sema F. Oktug. “Analysis of multi-player online game traffic based on self-similarity”. In: *Proceedings of 5th ACM SIGCOMM Workshop on Network and System Support for Games. NetGames ’06*. Singapore: Association for Computing Machinery, 2006, 25–es. ISBN: 1595935894. DOI: 10.1145/1230040.1230093. URL: <https://doi.org/10.1145/1230040.1230093>.
- [4] *Postmortem of Service Outage at 3.4M CCU*. <https://www.fortnite.com/news/postmortem-of-service-outage-at-3-4m-ccu>. 2018.
- [5] Ashwin R. Bharambe, Mukesh Agrawal, and Srinivasan Seshan. “Mercury: supporting scalable multi-attribute range queries”. In: *SIGCOMM Comput. Commun. Rev.* 34.4 (Aug. 2004), pp. 353–366. ISSN: 0146-4833. DOI: 10.1145/1030194.1015507. URL: <https://doi.org/10.1145/1030194.1015507>.
- [6] *Sony, IBM and Butterfly.net announce the activation of Linux-based computing grid*. <https://lwn.net/Articles/23968/>. 2003.
- [7] C. Greenhalgh and S. Benford. “MASSIVE: a distributed virtual reality system incorporating spatial trading”. In: *Proceedings of 15th International Conference on Distributed Computing Systems*. 1995, pp. 27–34. DOI: 10.1109/ICDCS.1995.499999.
- [8] Emmanuel Frécon and Mårten Stenius. “DIVE: a scaleable network architecture for distributed virtual environments”. In: *Distributed Systems Engineering* 5.3 (Sept. 1998), p. 91. DOI: 10.1088/0967-1846/5/3/002. URL: <https://dx.doi.org/10.1088/0967-1846/5/3/002>.
- [9] B. Knutsson et al. “Peer-to-peer support for massively multiplayer games”. In: *IEEE INFOCOM 2004*. Vol. 1. 2004, p. 107. DOI: 10.1109/INFCOM.2004.1354485.

- [10] Jens Müller, Andreas Gössling, and Sergei Gorlatch. “On correctness of scalable multi-server state replication in online games”. In: *Proceedings of 5th ACM SIGCOMM Workshop on Network and System Support for Games*. NetGames '06. Singapore: Association for Computing Machinery, 2006, 21–es. ISBN: 1595935894. DOI: 10.1145/1230040.1230088. URL: <https://doi.org/10.1145/1230040.1230088>.

A Experimental Parameters

The following settings were used to run our experiment: nodes run at a tickrate of 10 Hz, and clients send random directional inputs to each node at 2 Hz. Each client controls exactly one player, represented as a 2×2 square on a 100×100 grid. Players move at a speed of 0.005, and the radius of their area of interest is 3. Nodes have an artificial latency of 10ms per network call, and are restricted in total CPU usage using `cpulimit -l 10`.

The experiments can be run by first starting the locator service locally. The servers can be started manually or with `scripts/start-servers.sh`. Running the included `client.go` will automatically distribute and manage all players. The number of servers in the configuration as well as the total number of players must be manually adjusted or passed in as command-line arguments to the client, server, and locator.

In this project, all experiments were run in a Dockerized Alpine Linux environment running on an M3 Silicon Mac, with all services running on a single machine.